Using Poltergeist with Cucumber

In Cucumber, to get the JavaScript driver for a specific scenario you simply prefix the scenario with the @javascript tag:

```
@javascript
Scenario: We can change the order of tasks
```

Making the Test Pass

In this section you'll set up the JavaScript and present some passing code without unit testing. In the next chapter you'll set up JavaScript unit-testing tools and see how you might test-drive JavaScript features.

Rails 5.1 makes two important changes to the JavaScript environment:

- jQuery is no longer a default dependency.
- Webpack integration is available via the Webpacker gem.[2] [3]

Webpack is an optional replacement for the Rails asset pipeline and has the same general purpose: take files that might be written in JavaScript, CoffeeScript, TypeScript, Elm, or another language, and build them into JavaScript files that can be passed to the browser. Webpack's advantage is that it's flexible, it's tied into the current JavaScript ecosystem, and it allows for a much wider range of JavaScript and CSS tools to be built into your Rails project. The disadvantage is that Webpack is quite complicated, but the Webpacker extension written by the Rails team provides defaults that will allow you to get pretty far.

Even though jQuery isn't part of the Rails default, it's still probably the quickest way to handle our little feature here, so let's add it back in via Webpack. Rails uses a tool called Yarn to manage JavaScript package dependencies.[4] Yarn is the JavaScript equivalent of Bundler, and even has some of the same developers.

When you generated the Rails application you included Webpacker, but if you haven't yet, you can add it with this:

```
$ bundle exec rails webpacker:install
```

You can add jQuery to your dependencies with the following:

```
$ yarn add jquery
```

2. https://webpack.github.io
3. https://github.com/rails/webpacker
4. https://yarnpkg.com